Universidade Estadual de Campinas
Instituto de Computação

MO809 - Topics of Distributed Computing

2s2026

FINAL PROJECT REPORT

Cooperative Perception with DiscoNet Under Perfect Communication

---

Luis Alberto Vásquez Vargas

Professor: Allan M. de Souza

Presentation date: November 5, 2026

Luis Alberto Vásquez Vargas

Cooperative Perception with DiscoNet Under Perfect Communication

Final project report for the course MO809 - Topics of Distributed Computing.

Professor: Allan M. de Souza.

2s2026

Presentation date: November 5, 2026

Abstract

Cooperative perception combines observations from several vehicles to reduce the limitations of a single viewpoint, particularly when objects are occluded. This project studies DiscoNet, a model that encodes local LiDAR observations, aligns neighboring feature maps, and combines them through spatial attention before predicting car detections. The experiments use a released pretrained student network on V2X-Sim 2.0 under ideal communication and alignment, with no retraining or checkpoint selection.

The detection benchmark evaluates the same model with feature sharing disabled and enabled over 1,000 synchronized test frames and 4,000 active vehicle views. Cooperation increases Average Precision (AP) from 49.10% to 70.41% at an Intersection over Union (IoU) threshold of 0.5, and from 43.05% to 64.82% at 0.7. Because model weights remain fixed, this comparison measures the benefit of enabling information sharing in the evaluated student.

A separate runtime experiment measures complete model execution for 1–20 input slots on a fixed 30-frame cohort. Recorded scans are replayed above five inputs to increase computational load. A matched ablation then compares the original complete collaboration graph with a coarse-attention maximum spanning tree for 1–15 slots. The selected links restrict fine feature fusion while retaining bidirectional exchanges and self contributions. At 15 slots, the tree reduces median execution time by 46.4%, while AP decreases by 13.98 and 15.08 percentage points at IoU 0.5 and 0.7, respectively.

Together, the results show a detection benefit from feature sharing and a concrete runtime–accuracy trade-off when collaboration is made sparse. The analysis connects these measurements to DiscoNet's encoder, alignment, attention, and decoder stages, distinguishing the information available to each receiver from the computation required to process the group.

Keywords: Cooperative perception; LiDAR; DiscoNet; V2X-Sim; sparse collaboration; runtime scalability.

Contents

Abstract **i**

List of Figures **iii**

1 Introduction and Objectives **1**
1.1 Scope and Communication Assumptions . . . 2
1.2 Organization and Visual Material . . . 2

2 DiscoNet and Cooperative Feature Fusion **2**
2.1 Collaboration Strategies and Distillation . . . 2
2.2 From LiDAR Points to Occupancy Grids . . . 3
2.3 Reusable Units and the Encoder . . . 4
2.4 Spatial Alignment, Attention, and Fusion . . . 6
2.5 Pairing Features and Computing Edge Scores . . . 7
2.6 Spatial Weights and Feature Fusion . . . 8
2.7 Decoder, Skip Connections, and Detection Heads . . . 9
2.8 Pairwise Work . . . 10

3 Dataset and Experimental Protocol **10**
3.1 V2X-Sim Release and Evaluation Units . . . 10
3.2 Model, Hardware, and Software . . . 11
3.3 Detection Metrics and Postprocessing . . . 11
3.4 Confidence Scores and Retained Predictions . . . 12
3.5 Precision, Recall, and Average Precision . . . 12
3.6 Three Evidence Sets . . . 13

4 Full-Test Detection Benchmark **14**
4.1 Comparison and Results . . . 14
4.2 Fixed-Checkpoint Test Loss . . . 15
4.3 Qualitative Detection Example . . . 15

5 Runtime Scaling with Model Input Count **16**
5.1 Recorded Inputs and Replay Workloads . . . 16
5.2 Timing Boundary . . . 17
5.3 Results . . . 17

6 Original versus Tree Collaboration **18**
6.1 Coarse Attention and Maximum Spanning Tree . . . 18
6.2 Matched Timing and Accuracy Protocol . . . 19
6.3 Complete Results and Trade-Off . . . 20

7 Discussion and Limitations **20**

8 Conclusion **21**

A Reusable Blocks and Convolution Detail **23**
A.1 Callable Units and Shape Contracts . . . 23
A.2 The Alignment Code Example . . . 24
A.3 Numerical Input and Kernel Dimensions . . . 24
A.4 Conv2D: Accumulating Across Channels . . . 25
A.5 Conv3D: Accumulating Across Time and Channels . . . 26

References **29**

List of Figures

1 Complementary observations around an occluding truck . . . 1
2 The project's ideal communication assumptions . . . 2
3 Early, intermediate, and late fusion . . . 3
4 Teacher guidance and the deployed student . . . 3
5 From a local scan to occupancy height layers . . . 4
6 The reusable 2D unit and encoder stage . . . 5
7 Encoder tensor states and the collaboration branch . . . 5
8 Directed interactions and a spatial edge-weight map . . . 6
9 Aligning sender features to the receiver's grid . . . 6
10 A worked rotation and translation on one feature channel . . . 7
11 Pairing receiver and aligned sender features . . . 7
12 The edge-scoring subnetwork . . . 7
13 Normalizing spatial weights across senders . . . 8
14 Weighting contributions and summing the fused map . . . 9
15 Upsampling, skip concatenation, and the decoder block . . . 9
16 The complete decoder pipeline . . . 10
17 Worked intersection and union areas . . . 11
18 Three overlaps under the two matching thresholds . . . 12
19 A worked precision–recall curve and its envelope . . . 13
20 Recorded full-test precision–recall curves . . . 14
21 Recorded local and cooperative car detections . . . 15
22 Six synchronized camera views of the detection example . . . 16
23 Recorded runtime scaling from one to twenty slots . . . 17
24 Complete collaboration and the saved five-slot tree . . . 19
25 The three-frame, three-channel convolution input . . . 25
26 Spatial and temporal kernel slices . . . 25
27 The first Conv2D output and the complete spatial result . . . 26
28 Two temporal positions of the Conv3D window . . . 27

1 Introduction and Objectives

An individual vehicle observes only part of its surroundings. Buildings, other vehicles, sensor range, and viewpoint can hide objects that another vehicle can see. Cooperative perception addresses this limitation by combining information from multiple observers. The central question is which information to exchange and how to use it without making computation and communication costs impractical.

Figure 1 gives the motivating example used in the project presentation. Vehicle A follows a truck and obtains returns from its rear surface. Vehicle B has another line of sight and observes a car farther ahead. Combining observations makes that complementary information available to the receiver. This geometric example motivates cooperation; DiscoNet implements it by exchanging features extracted from the observations.

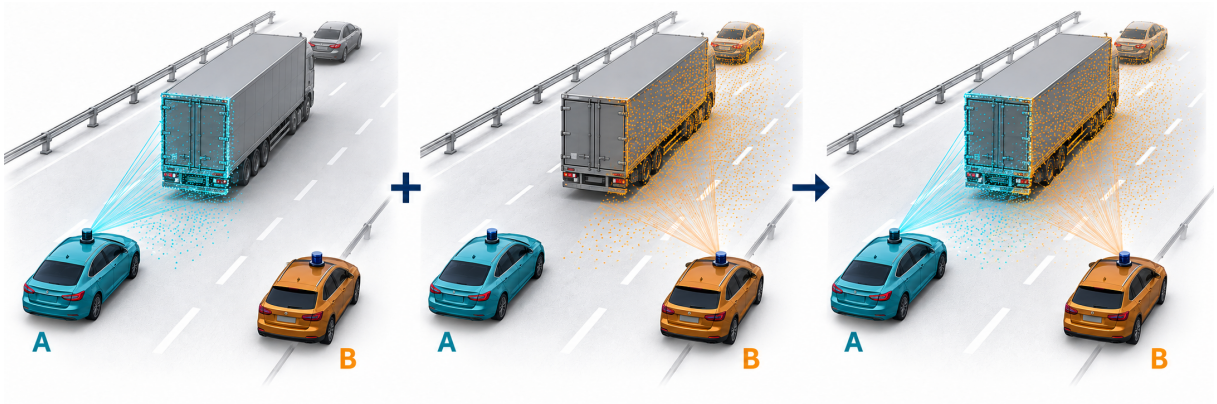


Figure 1: Project illustration of complementary LiDAR observations. Cyan and orange distinguish the two observers; the final panel combines their information. The gray car ahead of the truck is scene context in the first panel. This is the original AI-assisted illustration used in the presentation.

DiscoNet, introduced by Li et al. [3], learns spatially weighted collaboration between agents. This project examines that method through its architecture and recorded evaluations, rather than developing or training a detector from scratch. It addresses three questions:

1. How are individual LiDAR observations transformed into cooperative object detections?
2. How much does feature sharing improve detection for a fixed trained student under ideal communication?
3. How does execution time grow with the number of model inputs, and what changes when fine-resolution collaboration is restricted to a tree?

The original proposal planned a comparison of own-scan perception, early fusion, and DiscoNet. The completed evidence supports a narrower accuracy comparison: the same DiscoNet student is evaluated with information sharing disabled and enabled. Early fusion is explained as the teacher's training strategy, but no measured early-fusion baseline is available in these results. The sparse-tree experiment is an additional inference-time ablation of the released student; it introduces no new training or updated weights.

This study combines the architectural analysis and recorded evidence presented in the final project slides. It specifies the evaluation units and analyzes three distinct experiments. The full-test benchmark measures detection quality across all active vehicle views. A separate 30-frame workload measures runtime through 20 input slots. A matched 30-frame Original/Tree experiment measures runtime and ego-vehicle AP through 15 slots. These evidence sets are kept separate throughout.

1.1 Scope and Communication Assumptions

All conditions assume synchronized observations, accurate relative vehicle poses, no packet loss, and no communication delay. Agents are evaluated on a single computer with one GPU. Therefore, a model input slot represents an agent's data within that computation, not a physically distributed process. Measured runtime describes the implementation on the recorded hardware; it does not include a wireless network or establish an end-to-end vehicle communication latency.

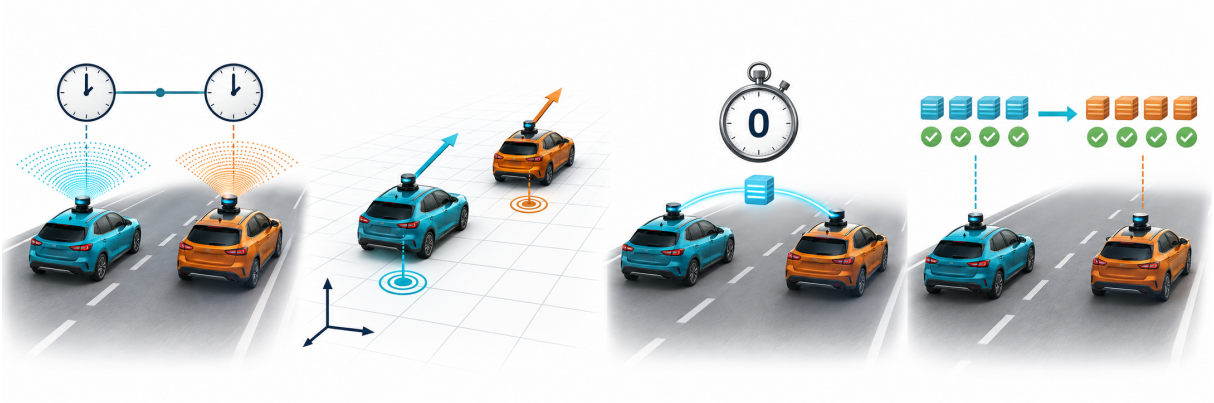


Figure 2: Project illustration of the four ideal conditions: synchronized observations, known poses, zero assumed transmission delay, and complete packet delivery. The clocks, poses, and packet symbols are explanatory artwork.

Figure 2 summarizes what perfect communication means in this study. Synchronization makes observations refer to the same instant, and relative poses allow their feature maps to be expressed in one coordinate system. Complete delivery means the model receives the selected inputs without dropped messages. Under these conditions, the experiments can concentrate on feature fusion and its computational cost.

1.2 Organization and Visual Material

The report follows the path from local observations to cooperative detections, then explains how those detections and their execution cost were evaluated. The architecture figures developed for the project expose individual tensor transformations. The worked metric and convolution examples make the numerical operations explicit. Recorded detection overlays, camera views, and quantitative plots then connect that explanation to the experiment evidence.

Existing project figures are reused throughout, including AI-assisted conceptual illustrations and vector diagrams. Captions identify teaching examples and recorded results. The final presentation, original figures, experimental records, and document sources are available in the project repository [4].

2 DiscoNet and Cooperative Feature Fusion

2.1 Collaboration Strategies and Distillation

Early fusion combines raw observations before feature extraction. Intermediate fusion exchanges learned features, while late fusion combines detection outputs such as boxes and scores. These choices place collaboration at different stages of the perception pipeline.

Figure 3 makes the exchanged representation explicit. Early fusion places information from different observers together before encoding. Intermediate fusion lets each observer first encode

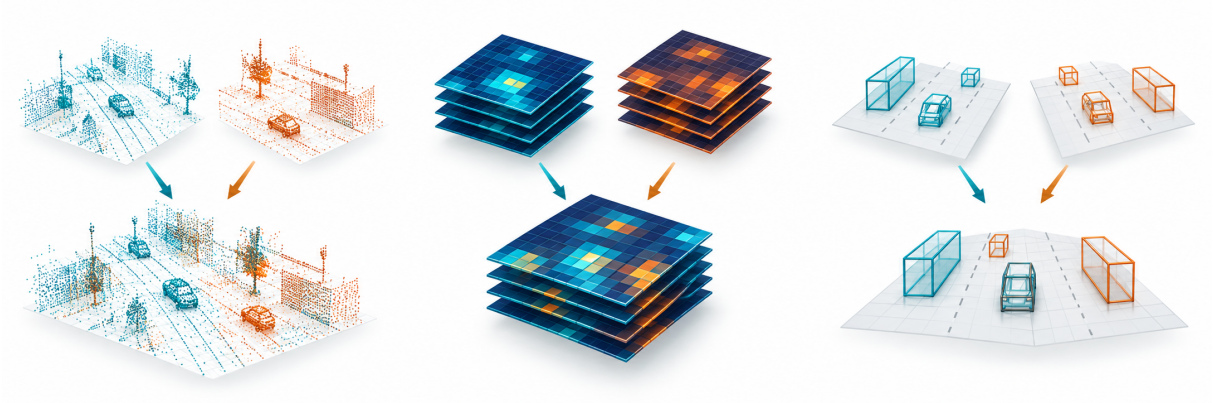


Figure 3: The project's comparison of raw-point, learned-feature, and detection-level combination. The colored tensors and boxes are conceptual AI-assisted artwork.

its own input. Late fusion moves collaboration after local predictions. This ordering determines which processing is local and which information is available when collaboration takes place.

DiscoNet uses an early-fusion teacher to guide an intermediate-fusion student during training. The teacher has a more complete observation, while the student receives local and shared features. Distillation encourages correspondence between their learned representations. Only the student is needed at inference [3]. The evaluations in this report load the released student checkpoint; the teacher and its distillation loss are not run as part of the measured detection pipeline.

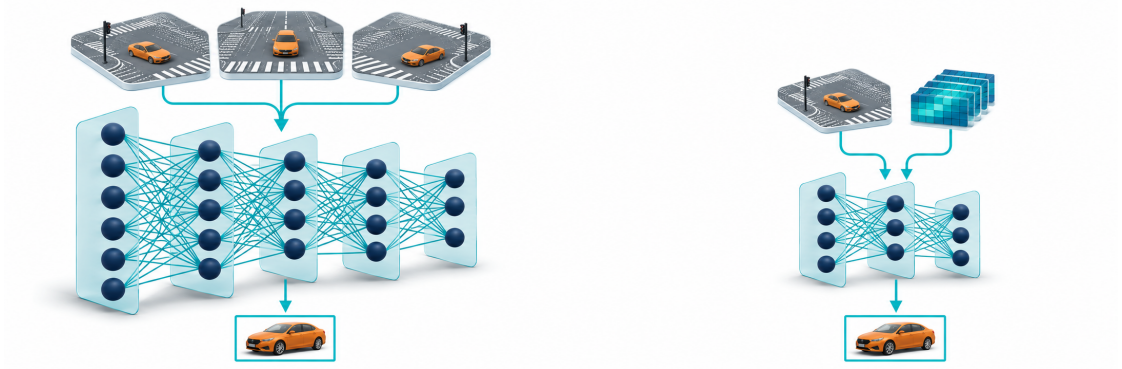


Figure 4: Project illustration of teacher guidance and the student's detection task. Network sizes express the teaching roles; the implemented teacher and student share the encoder/decoder backbone.

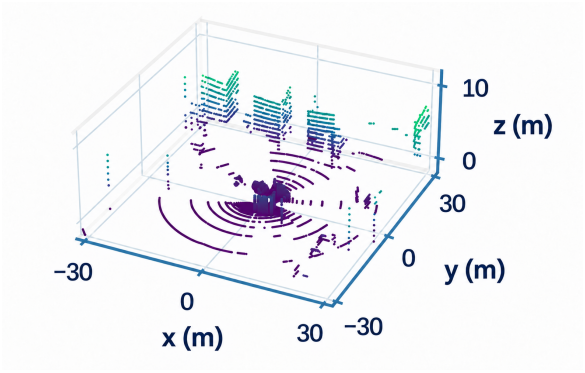
The training relationship in Figure 4 is distinct from the measured inference comparison. Turning sharing off restricts the same student to its local scan. Turning sharing on provides peer features without changing the checkpoint. The report therefore evaluates what collaboration adds to that fixed student, rather than comparing a large teacher with a separately trained smaller detector.

2.2 From LiDAR Points to Occupancy Grids

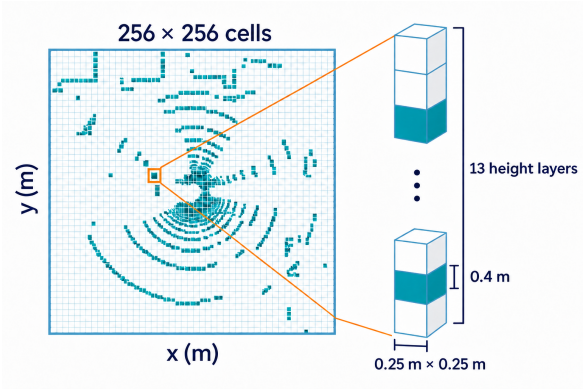
The detector consumes the authors' prepared V2X-Sim 2.0 detection data. CoPerception documents the preprocessing requirement and the availability of prepared inputs [1]. The weekly technical material describes the corresponding voxelization operation. Invalid coordinates are rejected, and points near the sensor origin are removed when both $-1 < x < 1$ and $-1 < y < 1$. Retained points satisfy

$$-32 < x < 32, \qquad -32 < y < 32, \qquad -3 < z < 2,$$

with distances in meters. A binary occupancy grid uses horizontal cells of 0.25×0.25 m and height bins of 0.4 m. The recorded model input has shape $256 \times 256 \times 13$ in height–width–channels order: occupied cells are set to one and cells without retained returns remain zero. The reported runs start from prepared grids; raw-point preprocessing is not part of their forward-runtime measurements.



(a) A local LiDAR observation



(b) Occupancy organized into height layers

Figure 5: Project preprocessing illustrations. Returns are assigned to spatial cells and height bins; occupied cells have value one. These conceptual AI-assisted views explain the representation rather than plotting a recorded input tensor.

The horizontal grid covers the 64-meter square around the sensor. Its thirteen channels represent vertical occupancy bins; they are not thirteen cameras or time steps. Figure 5 explains how height information can enter a network that applies 2D spatial convolutions. The layer stack is the channel dimension of the input feature tensor.

The diagrams use $H \times W \times C$ to make spatial size and channel count visible. PyTorch stores 2D feature tensors in $N \times C \times H \times W$ order. Batch and agent axes are omitted in the illustrated shapes. These different display conventions describe the same representation.

2.3 Reusable Units and the Encoder

The architecture is assembled from a reusable 2D unit: a 3×3 convolution followed by batch normalization and ReLU. An Initial unit can change channels while preserving spatial resolution; Keep preserves both; Down uses stride two and doubles the channels. Figure 6 shows the unit and the encoder stage that composes these operations.

The optional 3D helper inserts a singleton depth axis, applies a $1 \times 1 \times 1$ convolution, and flattens that axis again. Here it mixes channels at each location rather than aggregating multiple recorded frames. Appendix A contrasts this helper with a genuine temporal convolution and gives the callable block definitions.

The encoder computes multiscale features independently for each vehicle. Table 1 summarizes the tensor routing shown in the presentation. Batch and agent axes are omitted. The Stage 3 map is shared with DiscoGraph, while the deeper Stage 4 output remains a local decoder input. Saved encoder maps supply the decoder's skip connections.

Spatial resolution falls as channel count increases in Figure 7. Collaboration acts on a 32×32 grid, with 256 channels at each location, rather than directly on the 256×256 occupancy grid. The local Stage 4 path is retained alongside this shared representation. The decoder will use both.

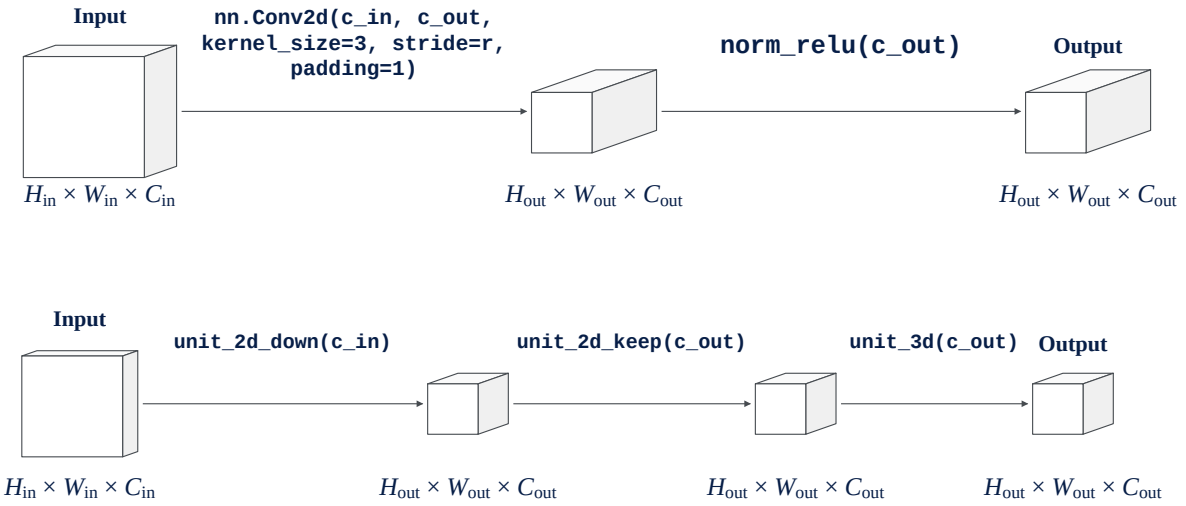


Figure 6: Project tensor diagrams: the 2D unit above and its use inside an encoder stage below. The optional pointwise 3D helper preserves the displayed spatial and channel dimensions.

Table 1: Encoder feature shapes and routing. Tensor order is $H \times W \times C$.

Stage	Output shape	Use
Input	$256 \times 256 \times 13$	Prepared occupancy grid
0	$256 \times 256 \times 32$	Saved for decoder Stage 8
1	$128 \times 128 \times 64$	Saved for decoder Stage 7
2	$64 \times 64 \times 128$	Saved for decoder Stage 6
3	$32 \times 32 \times 256$	Aligned and fused by DiscoGraph
4	$16 \times 16 \times 512$	Local decoder input

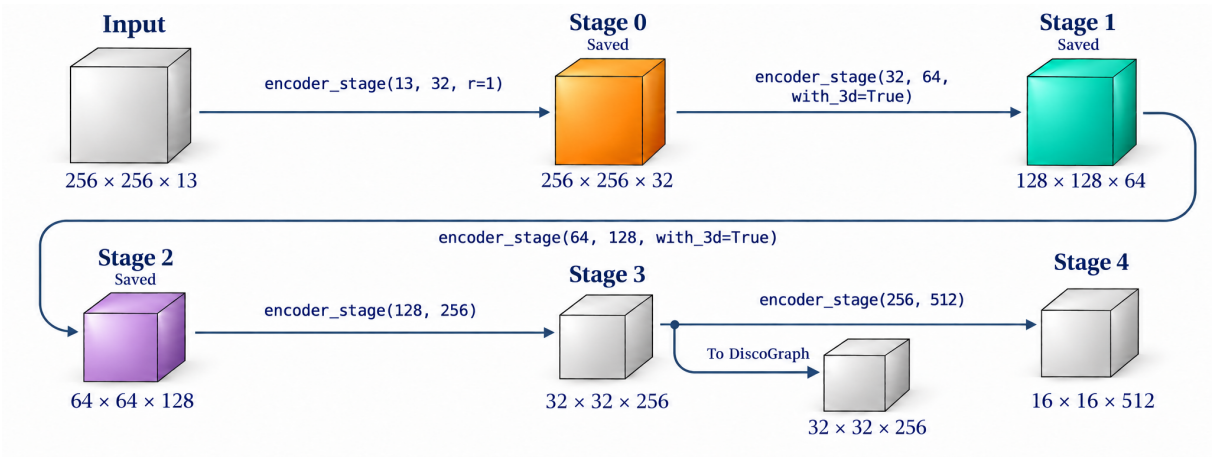


Figure 7: Project encoder pipeline illustration. Saved Stage 0–2 maps return through skip connections. Stage 3 supplies the shared representation; Stage 4 supplies the deeper local representation.

2.4 Spatial Alignment, Attention, and Fusion

Let F_u be sender u's feature map and let v be the receiving vehicle. An alignment operator $\mathcal{T}_{u \to v}$ rotates and translates the sender's map into the receiver's coordinates. It is based on their relative poses, so the two maps describe the same physical region. In the recorded implementation, feature resampling uses bilinear interpolation, zero padding, and `align_corners=False`.

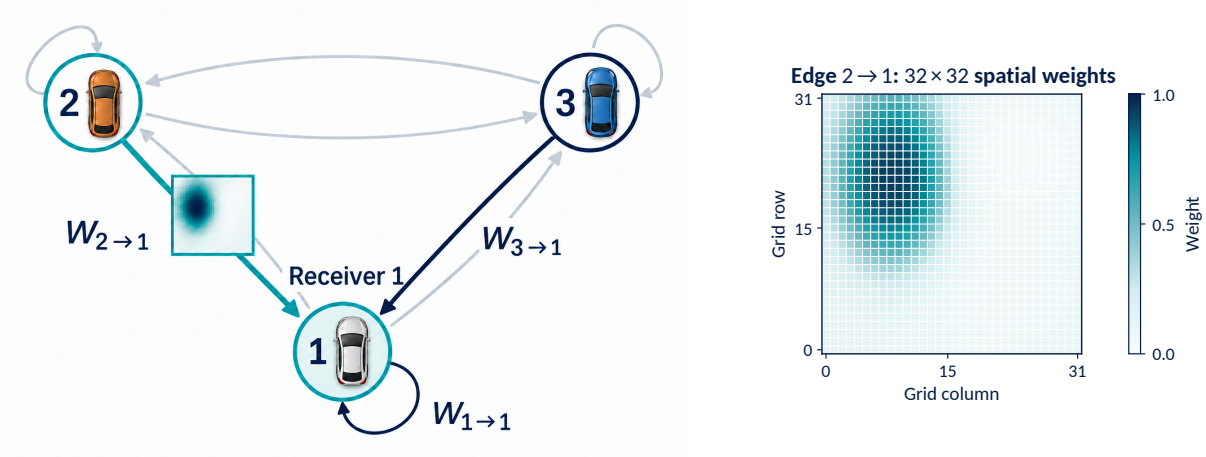


Figure 8: The project's three-agent teaching example. An edge carries a spatial weight map, so a sender's influence can vary across the receiver's grid. Displayed weights are synthetic examples.

For $u \to v$, receiver v defines the coordinate system. Reversing the direction changes both the receiving grid and the feature pair being scored. Equal array indices in two unaligned maps can refer to different physical locations. Figure 9 shows why corresponding locations must be brought into agreement before their features are paired.

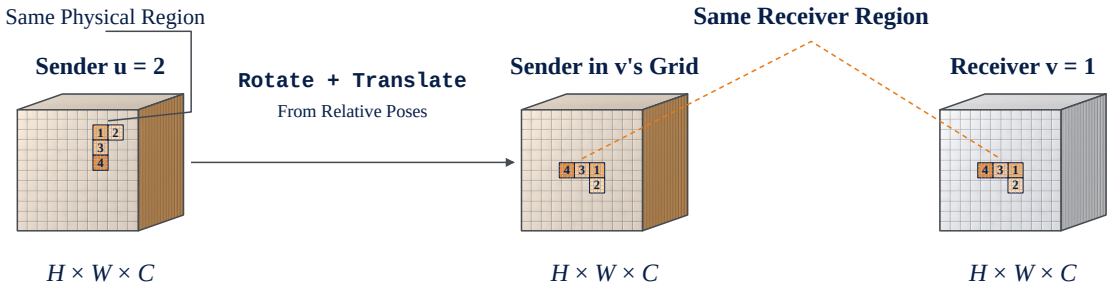


Figure 9: Project alignment diagram. A pose-based transform moves the sender's representation into the receiver's grid. The same spatial transform is applied to every channel.

The small example in Figure 10 makes the operation visible on one channel. Rotation changes the pattern's orientation; translation changes its destination position. Feature channels remain separate during this resampling. The sampling grid specifies where each output cell reads the input, so output-to-input sampling uses the inverse of a forward motion of points. The project's code example is included in Appendix A.

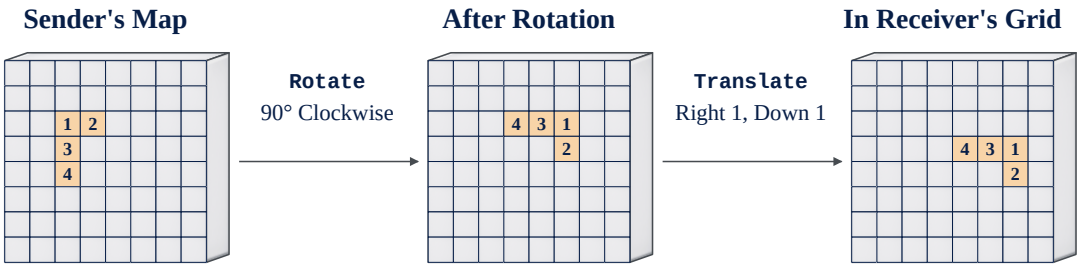


Figure 10: The project's 8×8 teaching example separates rotation from translation. Colored cells track how a spatial pattern is relocated before fusion.

2.5 Pairing Features and Computing Edge Scores

The aligned sender features and receiver features are concatenated across channels. A learned scoring function produces a spatial edge-score map,

$$s_{u \to v}(r, c) = g([F_v, \mathcal{T}_{u \to v}(F_u)])_{r,c}\,.$$

The implementation walkthrough reduces the paired channels through 128, 32, and 8 intermediate channels before producing one score per spatial cell.

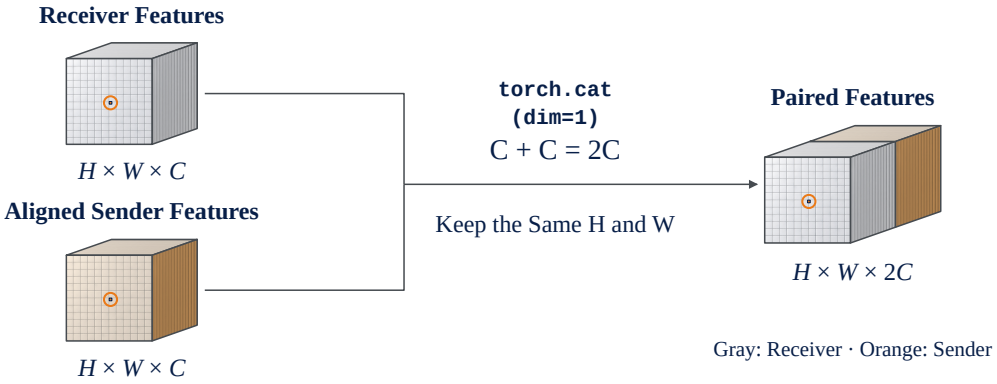


Figure 11: Project diagram of channel concatenation. Two 256-channel maps form a 512-channel pair at each of the 32×32 collaboration locations.

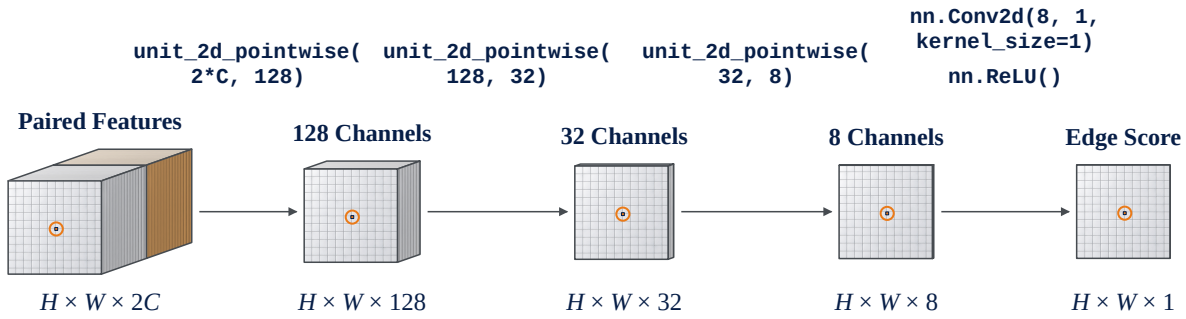


Figure 12: Project channel reductions from paired features to an $H \times W \times 1$ edge-score map. Spatial dimensions remain unchanged while pointwise convolutions reduce the channel count.

Concatenation preserves both input feature vectors rather than averaging them. The pointwise scoring network then mixes their channels at each location. Its final scalar describes that sender–

receiver interaction at one cell; it is distinct from the car-class confidence emitted later by the detection head.

2.6 Spatial Weights and Feature Fusion

A softmax across senders, including the receiver itself, gives

$$a_{u\to v}(r,c) = \frac{\exp s_{u\to v}(r,c)}{\sum_{j=1}^{n} \exp s_{j\to v}(r,c)}, \qquad \sum_{u=1}^{n} a_{u\to v}(r,c) = 1. \tag{1}$$

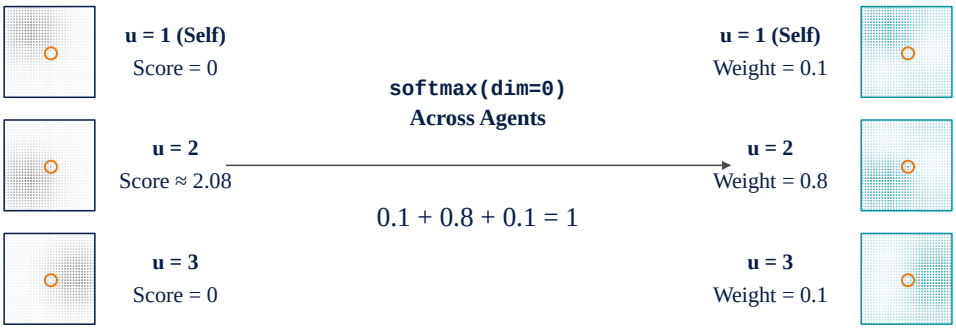


Figure 13: Project softmax illustration. For one receiver and one cell, all contributing sender weights sum to one. The operation is repeated across the spatial grid.

Normalization acts on the sender axis, not on feature channels or on all spatial cells together. Weights at two cells can therefore differ for the same edge. A sender can have more influence in one region than another, as the spatial map in Figure 8 illustrates.

The fused map is

$$\widetilde{F}_v(r,c,:) = \sum_{u=1}^{n} a_{u\to v}(r,c)\, \mathcal{T}_{u\to v}(F_u)(r,c,:). \tag{2}$$

Thus, a scalar weight at a cell multiplies its entire channel vector, and the weighted contributions are summed. The illustrated three-agent weights in the presentation are teaching examples, not attention values measured from these runs. Actual benchmark groups contain three to five active vehicles.

The fused output retains $H \times W \times C$. More senders add terms to the sum rather than adding output channels. The self contribution keeps the receiver's own representation in the same normalized competition as its peer contributions. This completes one collaboration round.

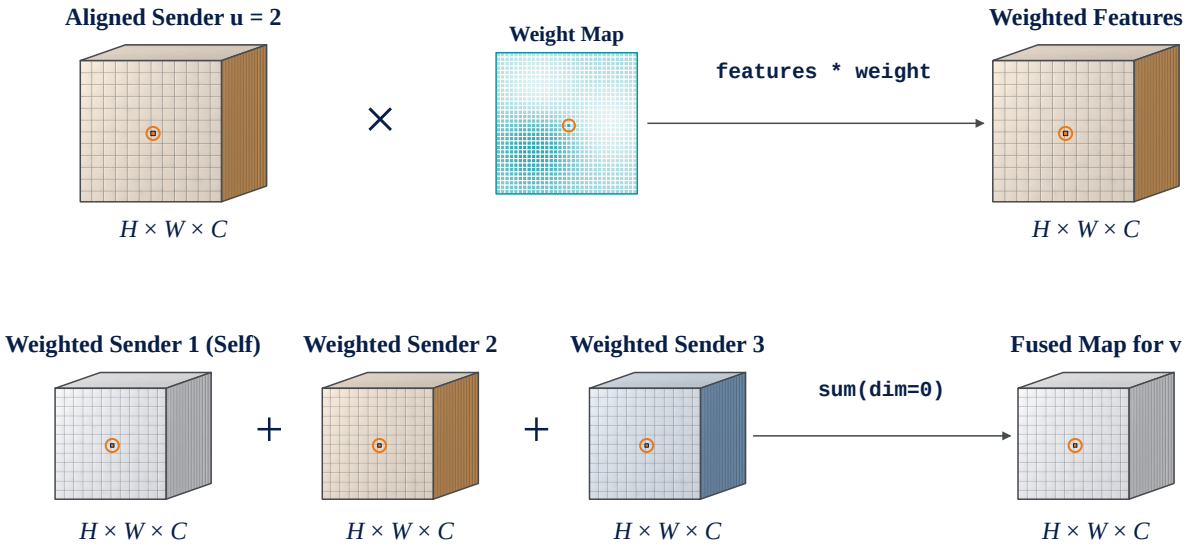


Figure 14: Project diagrams of the final operations. Above, a spatial scalar is broadcast over the channel vector. Below, weighted maps, including the self contribution, are added to form one fused map for the receiver.

2.7 Decoder, Skip Connections, and Detection Heads

The first decoder stage upsamples the local Stage 4 output and concatenates it with the fused Stage 3 map. Upsampling uses nearest-neighbor interpolation; concatenation joins channels while matching spatial dimensions. Thus a $16 \times 16 \times 512$ map becomes $32 \times 32 \times 512$ and is joined with $32 \times 32 \times 256$ fused features, giving 768 channels. Figure 15 exposes that sequence.

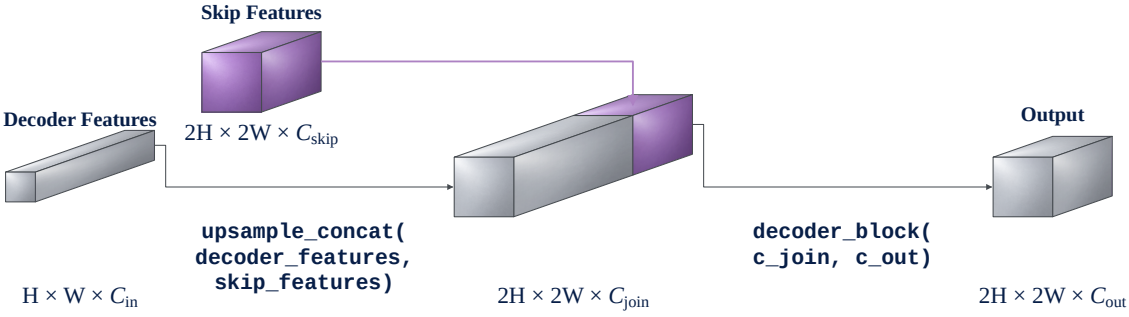


Figure 15: Project decoder-stage diagram. Upsampling changes spatial size, concatenation appends the skip channels, and the decoder block produces the next map.

Later stages use saved Stage 2, 1, and 0 features. Each skip already has the target spatial resolution. The decoder block contains two 2D units with stride one. Table 2 records the channel arithmetic, and Figure 16 shows the full route to the input grid size.

The final $256 \times 256 \times 32$ map feeds classification and box-regression heads. Classification produces background/car logits for candidate anchors; regression produces box parameters. Decoding, confidence filtering, and rotated NMS turn these dense outputs into the retained detection set. Section 3.3 explains its evaluation.

Table 2: Decoder routing. All shapes use $H \times W \times C$.

Stage	Skip source	Concatenated shape	Output shape
5	Fused Stage 3	$32 \times 32 \times 768$	$32 \times 32 \times 256$
6	Saved Stage 2	$64 \times 64 \times 384$	$64 \times 64 \times 128$
7	Saved Stage 1	$128 \times 128 \times 192$	$128 \times 128 \times 64$
8	Saved Stage 0	$256 \times 256 \times 96$	$256 \times 256 \times 32$

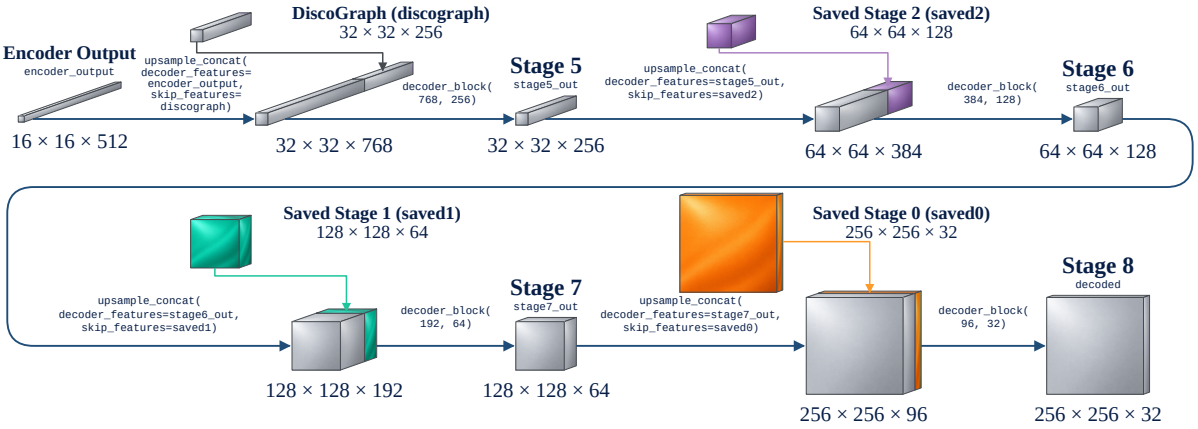


Figure 16: Project diagram of decoder Stages 5–8. The first skip is the fused collaboration map; the other three are saved local encoder maps. The output returns to 256×256 resolution.

2.8 Pairwise Work

For n active slots, the original complete graph evaluates n^2 ordered sender–receiver pairs when self pairs are counted. There are $n(n-1)$ peer pairs excluding self links. These are two conventions for the same graph, not conflicting complexity claims. Pairwise collaboration contributes quadratic work in n; the encoder, decoder, and heads also contribute to the measured full-group execution time.

3 Dataset and Experimental Protocol

3.1 V2X-Sim Release and Evaluation Units

V2X-Sim provides simulated multi-agent observations and annotations for collaborative perception [2]. The source recordings include vehicle and roadside sensors, but the experiments here exclude roadside-unit inputs. Cameras provide visual context; they are not inputs to this LiDAR detector. The official scene partitions and the recorded vehicle-only counts are summarized in Table 3.

Table 3: V2X-Sim 2.0 scene partitions used by the released detection protocol.

Partition	Scenes	Frames	Vehicle inputs/frame	Vehicle scans
Train	80	8,000	2–5	29,200
Validation	10	1,000	3–5	4,000
Test	10	1,000	3–5	4,000
Total	**100**	**10,000**	—	**37,200**

One synchronized frame contains observations from several active vehicles. One *vehicle view* is the output of one active vehicle at one frame. The full test split contains 300 frames with three vehicles, 400 with four, and 300 with five, giving 4,000 active views. Matching is performed within each view; predictions are then pooled for AP. An object observed by two vehicles contributes a reference box in each vehicle's evaluation view. The 50,047 reference boxes in the benchmark therefore count view-specific references, not unique physical cars across the recordings. The views are also correlated observations, not 4,000 independent synchronized frames.

The original DiscoNet paper evaluates V2X-Sim 1.0, whereas this study uses the prepared 2.0 detection release. CoPerception's documentation identifies 2.0 as its supported release [1]. The results are an evaluation of that released implementation and checkpoint, not an exact reproduction of the paper's 1.0 benchmark numbers.

3.2 Model, Hardware, and Software

All three experiments use the unchanged epoch-100 DiscoNet student in float32 precision on one NVIDIA GeForce RTX 5060 Ti with approximately 16 GB of GPU memory. The recorded setup has an AMD Ryzen 7 5700X CPU with eight cores and sixteen threads, approximately 16 GB of system RAM, and Omarchy Linux. The experiment environment records Python 3.14.8, PyTorch 2.9.1+cu128, and CUDA runtime 12.8. The following results use this recorded configuration.

The checkpoint contains 15,843,505 parameters and is loaded with a strict state-dictionary check. The inference implementation is pinned to source commit `dd8fbb660300`. Its checkpoint hash and run identities are stored with the evaluation records in the project repository. No training, checkpoint selection, or parameter update is performed in the reported evaluations.

3.3 Detection Metrics and Postprocessing

Intersection over Union measures overlap between a prediction B_p and its reference box B_g:

$$\mathrm{IoU}(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}.$$

AP is reported at matching thresholds of 0.5 and 0.7. The larger threshold requires more accurate localization.

Figure 17: Project IoU example. Both rectangles have area eight, their intersection has area six, and their union has area ten. The geometry is illustrative.

Figure 17 gives a concrete calculation:

$$\mathrm{IoU} = \frac{6}{8 + 8 - 6} = \frac{6}{10} = 0.60.$$

That prediction qualifies at a 0.5 matching threshold but fails at 0.7. Figure 18 extends the comparison to three overlaps. The reference box is held fixed while the displacement of the prediction changes. A stricter matching threshold asks for closer agreement with the reference geometry.

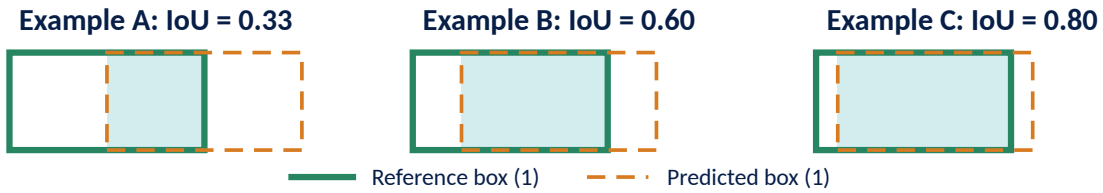


Figure 18: Project geometry examples with IoU approximately 0.33, 0.60, and 0.80. Only the last two qualify at 0.5; only the last qualifies at 0.7.

3.4 Confidence Scores and Retained Predictions

The retained car score is obtained by a softmax of background and car logits:

$$p_{\text{car}} = \frac{e^{z_{\text{car}}}}{e^{z_{\text{background}}} + e^{z_{\text{car}}}}.$$

The recorded evaluation keeps confidence strictly greater than 0.7 and uses rotated nonmaximum suppression (NMS) with IoU 0.01. Confidence filtering removes low-scoring candidates; NMS suppresses overlapping candidate predictions. These operations precede matching to reference boxes.

Table 4: The project's illustrative logits-to-confidence example.

Box	Background logit	Car logit	Car score	Score > 0.7
A	1.0	3.0	0.881	Keep
B	2.0	1.0	0.269	Discard
C	1.0	4.0	0.953	Keep

For Box A in Table 4, the conversion is $e^3/(e^1 + e^3) \approx 0.881$. Box C ranks above A because its car score is higher. Box B is removed by the confidence filter. These logits are the worked example from the project slides, rather than outputs selected from the benchmark.

Three thresholds have different jobs. The confidence threshold acts on a class score. The NMS IoU threshold compares predictions with other predictions. The evaluation IoU thresholds compare retained predictions with reference boxes. Thus the score threshold of 0.7 and matching threshold of 0.7 do not describe the same test.

3.5 Precision, Recall, and Average Precision

Predictions are ranked by their car-class confidence, matched one-to-one within their vehicle view, and classified as true positives (TP) or false positives (FP). Duplicate and unmatched predictions are false positives. Precision and recall are

$$P = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad R = \frac{\text{TP}}{N_{\text{reference}}}.$$

AP integrates the interpolated precision envelope as recall increases. It combines detection coverage and false-positive control; prediction count alone is not AP.

Table 5 and Figure 19 reproduce the project's four-prediction example with three reference cars. The first prediction matches a reference. The second is a false positive, so recall stays at 1/3 and precision falls to 1/2. The last two predictions match the remaining references. Each reference can be credited only once.

Table 5: Worked AP example with three reference cars and IoU matching at 0.5.

Rank	Score	Result	Cumulative TP / FP	Precision	Recall
1	0.99	TP	1 / 0	100.0%	33.3%
2	0.96	FP	1 / 1	50.0%	33.3%
3	0.92	TP	2 / 1	66.7%	66.7%
4	0.83	TP	3 / 1	75.0%	100.0%

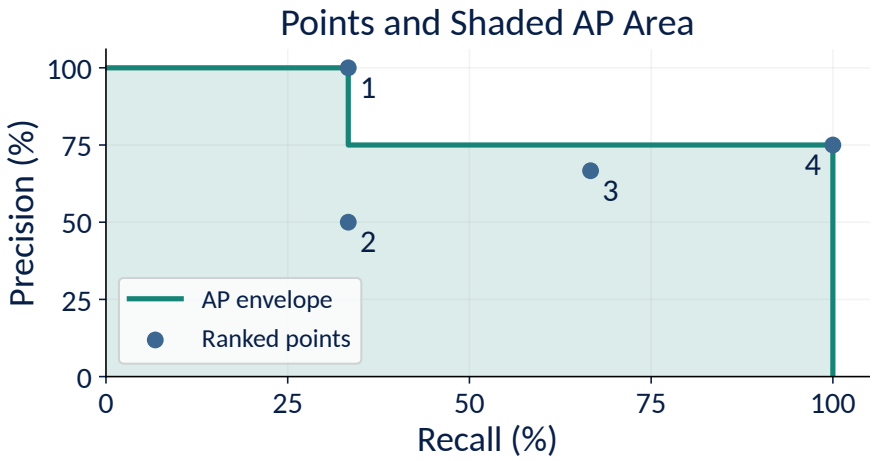


Figure 19: The project's illustrative AP curve for Table 5. The precision envelope uses the best precision available at or beyond each recall level.

The interpolated envelope has precision one over the first third of recall and 0.75 over the remaining two thirds. Its area is

$$\mathrm{AP} = \tfrac{1}{3} \times 1 + \tfrac{1}{3} \times 0.75 + \tfrac{1}{3} \times 0.75 = \tfrac{5}{6} \approx 83.3\%.$$

The false positive is present in the ranking even though every reference eventually receives a match. Conversely, missed or filtered-out references limit the maximum recall. The full-test curves in Section 4 apply this evaluation to the recorded population rather than this small teaching example.

3.6 Three Evidence Sets

The full-test experiment compares sharing off and on over all 1,000 test frames. The runtime-only experiment uses 30 fixed frames and measures $n = 1, \ldots, 20$ slots without AP. The Original/Tree comparison uses 30 fixed ego-1 views and measures both runtime and AP for $n = 1, \ldots, 15$. The latter two use ten frames each from raw scenes `scene_9`, `scene_97`, and `scene_98`. Their separately recorded timings must not be substituted for one another. AP from the ego-only ablation is also not directly comparable to AP pooled over all full-test vehicle views.

Table 6 gives the denominator for each result before the numerical comparisons. A synchronized frame, a vehicle view, an input slot, and a timed forward are different units. Repeating a forward yields another runtime sample, while repeating a scan in another slot changes the computational workload. Neither operation creates a new independent reference scene.

Table 6: The three experiment populations and the questions they answer.

Evidence set	Population	Reported outputs
Full test	1,000 synchronized frames; 4,000 active vehicle views	AP and fixed-checkpoint loss with sharing off/on
Runtime sweep	30 recorded frames; 1–20 input slots	Full-group forward median and P95; no AP
Original/Tree	30 ego-1 views; 427 reference cars; 1–15 slots	Matched forward medians and ego-view AP for both methods

4 Full-Test Detection Benchmark

4.1 Comparison and Results

The benchmark evaluates the same pretrained student twice. In *own scan only*, a vehicle's prediction uses only its local observation and sharing is disabled. In *cooperative perception*, the student also uses aligned features from the other active vehicles. Holding weights fixed isolates the effect of enabling collaboration in this model. It does not compare two separately trained detectors, and own scan only is not the paper's separately trained lower-bound baseline.

Table 7 contains the complete-split results from the saved benchmark. Cooperation raises AP by 21.31 percentage points at IoU 0.5 and 21.77 points at IoU 0.7. Improvements at both thresholds show that the gain is not restricted to lenient box overlap in this evaluation.

Table 7: Detection AP on 1,000 test frames and 4,000 active vehicle views. Both conditions use the same epoch-100 weights.

Condition	AP@0.5 (%)	AP@0.7 (%)
Own scan only	49.10	43.05
Cooperative perception	70.41	64.82
Gain (percentage points)	**21.31**	**21.77**

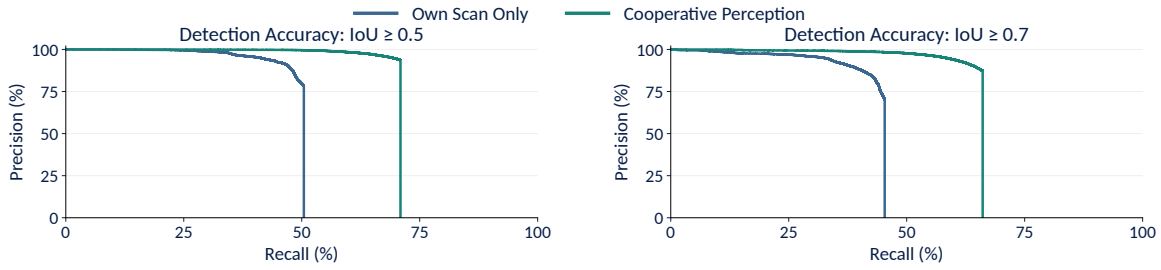


Figure 20: Recorded full-test precision–recall curves at IoU 0.5 and 0.7. The curves and plotted values are reused from the saved evaluation, without new inference or redrawing of the measured data.

The reproduced cooperative scores are approximately 70.4120% and 64.8237%. The archived audit compares these with the maintainer's checkpoint log, 70.4136% and 64.8239%, respectively; the discrepancies are below 0.002 percentage points. The documentation table instead lists 69.03% and 63.44%. Agreement with the checkpoint log should not be described as agreement with every published reference table, nor as replication of the original paper's different-release benchmark.

The saved all-active-view accounting and the authors' collection policy give identical AP and omit no active views. The record also reports exact self-only output parity and selected

postprocessing parity checks. These checks support the implementation comparison, but do not establish robustness to other datasets, checkpoints, or communication conditions.

4.2 Fixed-Checkpoint Test Loss

The supplementary record reports classification, box-localization, and total test loss. Table 8 gives the unrounded-source values displayed to two decimals. The original test code sums loss across five padded vehicle slots, divides by five, and then averages across synchronized frames. Inactive slots contribute zero. Thus these values must be interpreted with that normalization rather than as loss per active vehicle or a count of incorrect detections.

Table 8: Mean fixed-checkpoint test loss under the authors' normalization. No teacher or distillation term is included during evaluation.

Condition	Classification	Localization	Total
Own scan only	2,978.19	13,844.24	16,822.43
Cooperative perception	1,528.45	9,579.85	11,108.29

All three loss values are lower with collaboration, consistent with the AP comparison. This is one evaluation of a fixed checkpoint, not a training curve, learning-speed comparison, or reconstruction of validation history. The loss and AP records therefore answer related but different questions.

4.3 Qualitative Detection Example

Figure 21 shows ego vehicle 1 in raw `scene_5`, frame `000066`, with five participating vehicles. This example comes from the recorded validation scene, not the 1,000-frame test benchmark. Within the ego crop, there are 17 reference car boxes. Own scan only produces 11 predictions, while cooperation produces 17. At IoU at least 0.5, the predictions match 10 and 16 reference cars, respectively. Equal prediction and reference counts in the cooperative case therefore do not mean perfect matching.

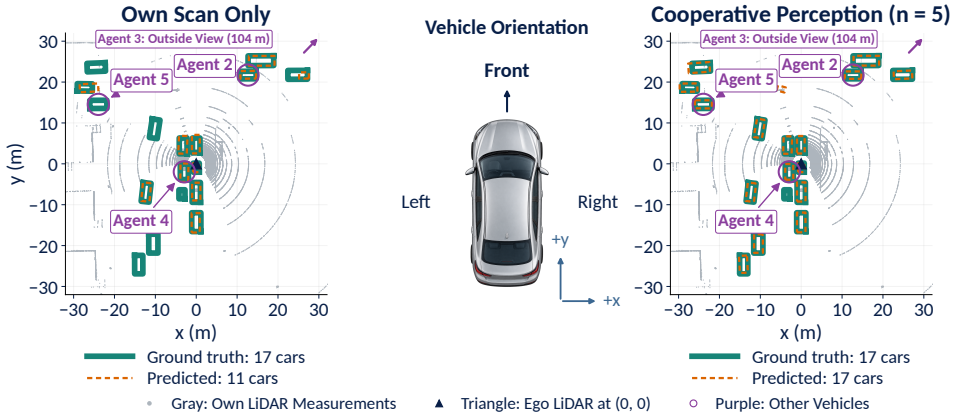


Figure 21: Recorded local and cooperative car detections in the single-frame example. Reference and prediction outlines remain overlaid. Axes use the vehicle-relative front-up display about the LiDAR sensor; the original model coordinates are unchanged. The orientation icon is illustrative AI artwork; points, reference boxes, and predictions are recorded data.

The example illustrates why sharing can help: another viewpoint can supply useful features where the receiver's own observation is incomplete. Its counts do not measure full-test AP or the performance of all five receivers. The camera views in the slide appendix provide synchronized

context, but camera and LiDAR positions differ and the model does not use those images as detector inputs.

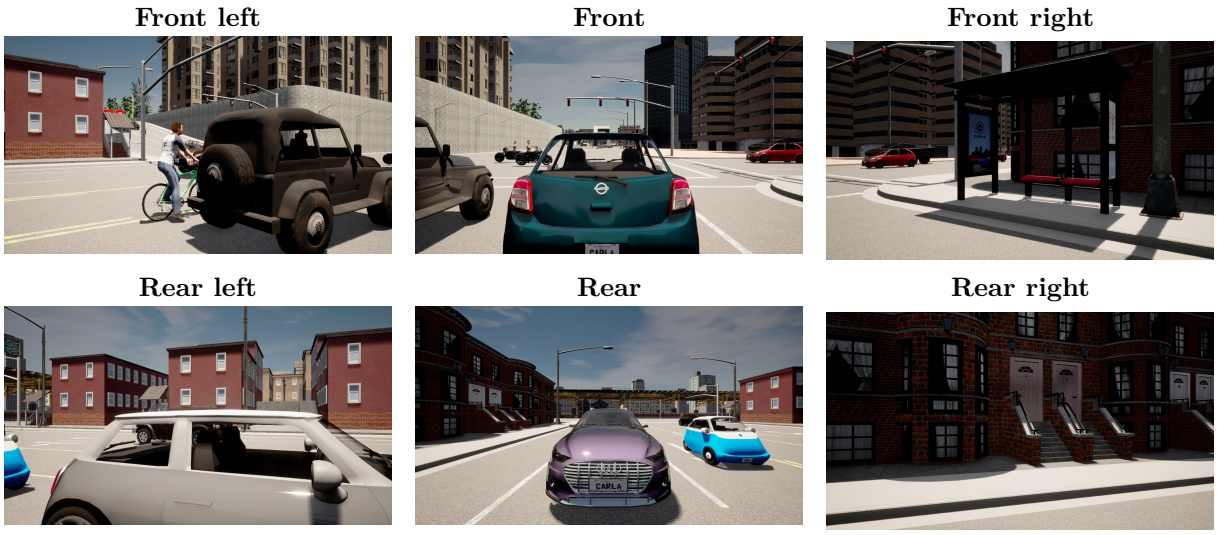


Figure 22: The six V2X-Sim 2.0 camera views archived with the project's `scene_5`, frame `000066` example. They show the surrounding road geometry and traffic from the ego vehicle.

Figure 22 provides the visual setting for the bird's-eye detection overlays. The views explain why a nearby vehicle and a farther reference box can have different visibility from the ego sensor. They also show why the reference-box set should not be equated with the returns visible in one scan. Camera and LiDAR mounting positions differ, so the camera panels supply scene context rather than a pixel-level projection of every LiDAR point.

The useful comparison in Figure 21 is therefore which reference boxes gain a sufficiently overlapping prediction when sharing is enabled, rather than whether the scene looks busier or the output contains more boxes. The 10-to-16 increase in matched references supports that interpretation for this example. Aggregate AP is still determined by the full ranked evaluation, including false positives and missed references.

5 Runtime Scaling with Model Input Count

5.1 Recorded Inputs and Replay Workloads

The runtime-only experiment fixes the checkpoint and a 30-frame cohort while varying n from one to twenty. Every frame provides five recorded vehicle inputs. For $n \leq 5$, ego 1 is kept and the nearest recorded vehicles are added. For $n > 5$, the same five scans and their recorded pairwise poses are cycled into additional tensors and active slots. For example, $n = 20$ contains four copies of the five recorded inputs.

Consequently, n denotes model input slots. Above five, this experiment tests computational load rather than perception from newly observed vehicles. No AP or box postprocessing is computed in this runtime-only sweep. Its role is to expose execution cost under increasing workload, not to predict detection gains with twenty independent vehicles.

Table 9 makes the workload increase explicit. The implementation allocates additional tensors and active slots even when the underlying scan is repeated. Encoding, pairwise collaboration, and decoding therefore process a larger group. Cycling recorded poses keeps each replay tied to its source observation, rather than placing a newly observed vehicle at an invented location.

Table 9: Input reuse and original fine-pair workload at selected slot counts.

Slots n	Distinct recorded inputs	Ordered pairs n^2	Input composition
1	1	1	Ego input
5	5	25	Five recorded inputs
10	5	100	Two cycles of the five inputs
20	5	400	Four cycles of the five inputs

5.2 Timing Boundary

Each condition uses the same 30 frames in three trials, with twenty warmup forwards per condition and trial. Condition order is shuffled with seed zero. This gives 90 timed samples per n, or 1,800 timed forwards and 1,200 warmup forwards across the sweep. Inputs are already on the GPU, and CUDA is synchronized before and after each measured forward.

The forward timer includes the encoder, pairwise fusion, decoder, and detection heads for the *entire n-slot group*. It excludes file reads, CPU packing, raw LiDAR preprocessing, input transfer, model loading, warmups, and output checks. These times are not per-vehicle wireless latency and are not the duration of the whole data-loading/evaluation pipeline.

5.3 Results

Median full-group forward time increases from 7.76 ms at $n = 1$ to 306.05 ms at $n = 20$, a factor of 39.4. Table 10 reports every condition, with the 95th percentile (P95) of its 90 timing samples. P95 describes the observed tail of those timings; it is not a confidence interval for the median.

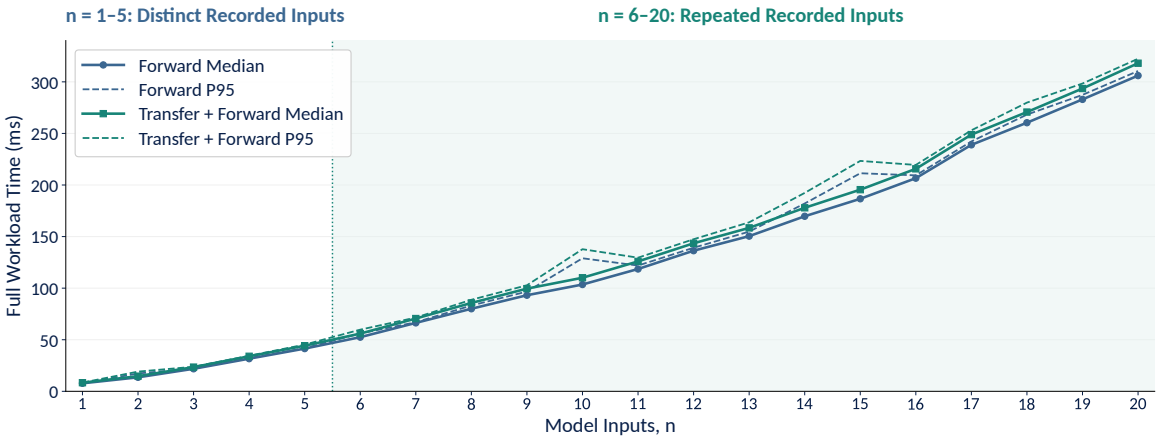


Figure 23: Saved runtime-scaling figure. Forward timings and transfer-plus-forward timings have different boundaries; only the former are tabulated in Table 10. The plot retains the original measured medians and P95 values.

The growth is consistent with the increasing cost of a pipeline containing all-pair feature collaboration. At $n = 20$, the original graph evaluates 400 ordered fine-resolution pairs including self pairs, compared with one at $n = 1$. The measured runtime ratio is not required to equal the pair-count ratio because other stages, batching, and hardware behavior also contribute. The experiment does not fit a universal complexity law or isolate fusion time from the rest of the model.

The two timing boundaries in Figure 23 are part of the experiment setup. The forward-only series matches the tabulated GPU-ready workload. The transfer-plus-forward series includes

Table 10: Runtime-only sweep on the fixed 30-frame cohort. The rule after $n = 5$ marks the change from distinct recorded inputs to replayed-input workloads. Timings refer to full-group forward execution.

n	Recorded inputs	Replayed inputs	Median (ms)	P95 (ms)
1	1	0	7.76	7.83
2	2	0	13.57	17.31
3	3	0	21.85	22.11
4	4	0	31.64	32.16
5	5	0	41.42	41.82
6	5	1	52.49	56.07
7	5	2	66.34	66.95
8	5	3	80.03	83.18
9	5	4	93.16	96.70
10	5	5	103.53	128.98
11	5	6	118.58	121.92
12	5	7	136.32	139.17
13	5	8	150.43	154.80
14	5	9	169.69	182.09
15	5	10	186.63	211.39
16	5	11	206.61	209.36
17	5	12	238.93	242.22
18	5	13	260.39	268.26
19	5	14	282.99	287.36
20	5	15	306.05	310.52

an additional input transfer stage. Neither includes all operations of the full-test evaluation, such as file access, prediction export, and AP calculation. Separating these boundaries avoids attributing the whole evaluation duration to the network's forward pass.

6 Original versus Tree Collaboration

6.1 Coarse Attention and Maximum Spanning Tree

The second custom experiment asks whether restricting expensive fine-resolution collaboration reduces execution time and how much detection quality is lost. The released weights are reused without retraining. The variant has three stages:

1. **Coarse scoring.** Each 32×32 feature map is pooled to 4×4. All ordered pairs are aligned and scored at this coarse resolution, with weights normalized over senders. Each directional attention map is reduced to its mean squared attention.

2. **Tree selection.** The two directional scores of an unordered vehicle pair are averaged. Descending Kruskal selection keeps the highest-scoring links that do not create a cycle, until $n - 1$ remain. This is a maximum spanning tree, rather than a minimum spanning tree.

3. **Fine fusion.** Each selected edge is used in both directions, and every self pair is retained. The original 32×32 alignment and scoring are applied only to these pairs. Attention is renormalized over the retained senders, and the normal decoder and heads follow.

For each receiver–sender pair, the coarse attention statistic can be written as

$$q_{u\to v} = \frac{1}{H_c W_c} \sum_{r,c} a^{\text{coarse}}_{u\to v}(r,c)^2, \qquad q_{\{u,v\}} = \frac{q_{u\to v} + q_{v\to u}}{2}, \quad H_c = W_c = 4.$$

The undirected tree determines which directed fine interactions are retained. Fine pair evaluations change from n^2 to

$$n + 2(n-1) = 3n - 2.$$

At $n = 15$, that is 225 original fine pairs versus 43 tree fine pairs, in addition to the tree variant's 225 coarse pairs. Only the fine-pair count becomes linear. Coarse scoring remains quadratic, and CPU Kruskal selection introduces additional work. This is not a claim of linear complexity for the complete algorithm.

There is one fusion round. A receiver can use its immediate tree neighbors, but a path in the spanning tree does not transmit every remote vehicle's features across multiple hops during that round. Sparsity therefore changes the available information as well as the amount of computation.

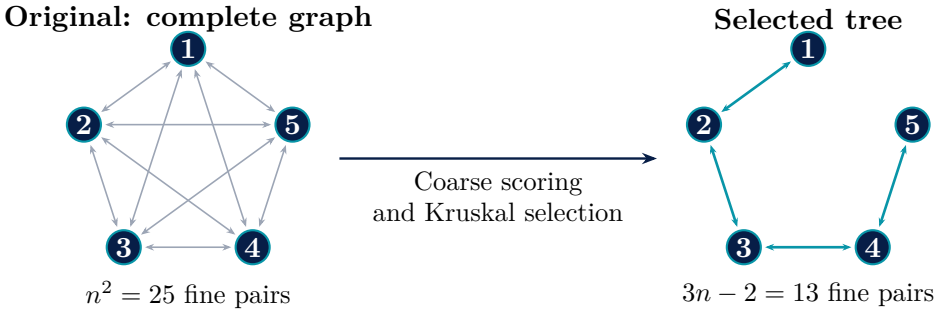


Figure 24: The project topology diagram adapted for the report, using the saved five-slot preflight tree. Double arrows denote interactions in both directions. Self links are counted in the computation but omitted from the drawing. The original has 25 fine pairs; the tree has 13 fine pairs plus coarse scoring.

Figure 24 gives a concrete example of what a receiver loses when edges are removed. In the displayed tree, node 1 has node 2 as its only peer neighbor. Its fine fusion can combine the self contribution and node 2, while the original graph can use all five inputs. Node 3 has two peer neighbors and can combine those with its own features. A path from node 1 to node 5 establishes graph connectivity, but it does not make node 5's original map an immediate input to node 1 in one fusion round.

The displayed topology is one saved preflight example. Selection is driven by the coarse attention scores of the evaluated input group; the figure does not assign a fixed chain to every frame. It illustrates the difference between retaining connectivity and retaining all direct fine observations.

6.2 Matched Timing and Accuracy Protocol

The comparison uses the same 30 ego-1 views from three recorded test scenes at every condition, with 427 reference car boxes counted once across the cohort. Both methods use the unchanged checkpoint, float32 precision, and the same retained-prediction settings. There are three timing trials and 90 samples per method and n, covering $n = 1, \ldots, 15$. The specification records twenty warmups per method/count/trial, shuffled condition order with seed zero, deterministic cuDNN, and TF32 disabled.

The timer includes the complete model, coarse scoring, CPU tree selection, fine fusion, decoder, and heads. Inputs are already on the GPU. Reads and packing, transfers, warmups, model loading, box decoding, NMS, and AP scoring are outside the forward timer. AP is evaluated

separately on the original ego-1 labels. The run contains 2,700 timed forwards in total, and recorded ego outputs are identical across timing trials for a given condition.

As in the runtime-only workload, $n \leq 5$ uses distinct recorded inputs, while larger n replays the five recorded scans and poses. AP above five describes the behavior of those repeated inputs, not the benefit of extra independently recorded viewpoints.

6.3 Complete Results and Trade-Off

Table 11 reproduces all fifteen measured conditions from the versioned CSV and checks them against the saved JSON during report builds. The AP values are percentages; differences between them are percentage points.

Table 11: Matched Original/Tree comparison on 30 ego-1 views. Below the rule after $n = 5$, additional slots replay the five recorded inputs. The table is separate from the full-test benchmark and the runtime-only sweep.

	Median time (ms)		AP@0.5 (%)		AP@0.7 (%)	
n	Original	Tree	Original	Tree	Original	Tree
1	7.82	7.86	48.91	48.91	45.54	45.54
2	13.34	13.25	60.94	60.94	53.67	53.67
3	21.99	21.41	66.52	60.17	58.71	52.34
4	31.39	29.38	70.80	58.06	63.39	50.37
5	41.57	36.54	74.46	62.65	67.53	53.66
6	52.43	43.63	73.54	57.71	66.61	50.69
7	66.67	52.89	73.93	57.87	66.70	50.57
8	79.83	60.47	73.80	59.16	66.90	52.01
9	92.07	67.48	74.50	61.56	67.06	53.33
10	102.80	70.12	74.46	61.72	67.53	53.66
11	118.76	76.93	74.00	58.57	67.05	50.44
12	133.49	82.84	74.17	58.77	67.20	50.72
13	147.97	86.52	74.36	59.50	67.18	51.61
14	165.79	92.73	74.90	60.93	67.50	52.75
15	185.05	99.25	74.46	60.48	67.53	52.45

At $n = 15$, median time falls from 185.05 to 99.25 ms, a 46.4% reduction. AP@0.5 decreases from 74.46% to 60.48%, a loss of 13.98 percentage points. AP@0.7 decreases from 67.53% to 52.45%, a loss of 15.08 points. This is a substantial runtime reduction accompanied by a substantial accuracy loss, rather than an improvement at unchanged detection quality.

The same trade-off is visible within the distinct-input region. At $n = 5$, median time falls from 41.57 to 36.54 ms, while AP@0.5 falls from 74.46% to 62.65% and AP@0.7 from 67.53% to 53.66%. Thus the accuracy penalty is not only an artifact of replaying inputs above five.

At $n = 1$ and $n = 2$, the original and tree graphs coincide, and the saved predictions are identical. Small differences in their medians are timing variation, not evidence of a useful speed or accuracy difference. At larger n, the original AP is not strictly monotonic, and replay does not create new information. No general claim that increasing slots must improve detection follows from the table.

7 Discussion and Limitations

The full-test comparison establishes that feature sharing benefits the evaluated pretrained student under the stated conditions. It holds model weights fixed and changes collaboration availability.

It does not determine the ranking of separately trained local, early-fusion, and intermediate-fusion models. In particular, the planned early-fusion baseline remains unmeasured.

The scaling experiments address a different question: whether complete collaboration is computationally expensive as the number of slots increases, and what happens when its fine interactions are restricted. The tree ablation shows a measured runtime–accuracy trade-off. Removing fine edges can reduce access to complementary observations in a single fusion round. This is a plausible explanation of the AP loss, consistent with the mechanism, rather than a measured causal decomposition of every missed detection.

Several limits constrain these conclusions:

- **Ideal communication and poses.** The experiments do not vary latency, packet loss, bandwidth limits, asynchronous sensing, or pose errors. Their forward times cannot be used directly as wireless latency.
- **Replay beyond five inputs.** Larger workloads increase active model slots without adding independent viewpoints or real vehicles.
- **Different evaluation populations.** The full-test AP pools 4,000 active views; the tree AP uses 30 fixed ego views. Their numerical levels are not interchangeable estimates of the same population.
- **One checkpoint and device.** No retraining, model-family comparison, multi-GPU experiment, or cross-hardware generalization is established. The sparse graph was not trained to compensate for missing fine links.
- **Limited uncertainty analysis.** Repeated timings characterize runtime variation on a fixed cohort. They do not create new independent detection samples, and no confidence interval for population AP is reported.
- **Version and scope.** The evaluation uses V2X-Sim 2.0, unlike the original paper's 1.0 benchmark. Fixed-checkpoint losses are not a training history or evidence about convergence.

Follow-up experiments could hold the evaluation population and postprocessing fixed while varying retained neighbors, selection resolution, and fusion rounds. Further recordings would allow study beyond five independent vehicles. Fine-tuning a sparse student and testing communication impairments would require new training and an explicit transport model, respectively.

8 Conclusion

DiscoNet combines per-vehicle feature extraction, pose-based alignment, spatial attention, and decoder skip connections to produce cooperative car detections. On the complete prepared V2X-Sim 2.0 test split, enabling sharing in the same student raises AP by 21.31 and 21.77 percentage points at IoU 0.5 and 0.7. This directly answers the completed own-scan/cooperative comparison under ideal conditions.

The separate workload sweep shows increasing full-group computation as model input count grows. The matched tree experiment reduces fine pair evaluations but also reduces detection quality: at 15 slots, median runtime is 46.4% lower, with AP losses of 13.98 and 15.08 points. The results support studying collaboration sparsity as a trade-off, while preserving the distinction between additional computation and additional independently observed information.

A Reusable Blocks and Convolution Detail

The slide appendix presents the architecture through callable PyTorch-style blocks. The following excerpt preserves that decomposition; it explains the forward structure rather than replacing the authors' implementation.

A.1 Callable Units and Shape Contracts

```
def norm_relu(c):
    return nn.Sequential(nn.BatchNorm2d(c), nn.ReLU())

def unit_2d(c_in, r, c_out):
    conv = nn.Conv2d(c_in, c_out, 3, stride=r, padding=1)
    return nn.Sequential(conv, *norm_relu(c_out))

def unit_2d_initial(c_in, c_out):
    return unit_2d(c_in, 1, c_out)

def unit_2d_keep(c_in):
    return unit_2d(c_in, 1, c_in)

def unit_2d_down(c_in):
    return unit_2d(c_in, 2, 2*c_in)

def decoder_block(c_in, c_out):
    return nn.Sequential(unit_2d_initial(c_in, c_out),
                         unit_2d_keep(c_out))

def upsample_concat(decoder_features, skip_features):
    upsampled_features = F.interpolate(
        decoder_features, scale_factor=2, mode="nearest")
    return torch.cat((upsampled_features, skip_features), dim=1)
```

For even input dimensions, an Initial or Keep unit preserves spatial resolution, while a Down unit halves height and width. More generally, the kernel-3, padding-1, stride-r convolution gives $H_{\text{out}} = \lfloor (H_{\text{in}} - 1)/r \rfloor + 1$, with the analogous formula for width. The attention pointwise unit instead uses a 1×1 convolution to change channels without mixing neighboring spatial cells. Normalization and activation preserve the tensor shape.

An encoder stage composes two 2D units and, where specified, a pointwise 3D unit. In this single-frame reconstruction, the 3D helper temporarily inserts a singleton depth axis, applies a $1 \times 1 \times 1$ Conv3d, and flattens that axis again. It preserves the displayed $H/W/C$ dimensions. Its name is not evidence of multi-frame temporal aggregation in this forward path.

A.2 The Alignment Code Example

The project illustrates spatial alignment with a square feature map and a rotation followed by translation. The excerpt below is the existing teaching implementation used beside Figure 10. It constructs a forward transform, inverts it for output-to-input sampling, and applies the same grid to every channel.

```
import math
from torch.nn import functional as F

def align_features(
    sender_features, angle_deg, dx, dy
):
    _, C, H, W = sender_features.shape
    a = math.radians(angle_deg)
    cos, sin = math.cos(a), math.sin(a)
    transform = sender_features.new_tensor([
        [cos, -sin, 2 * dx / W],
        [sin,  cos, 2 * dy / H],
        [0,    0,    1],
    ])
    inverse = transform.inverse()
    theta = inverse[:2].unsqueeze(0)
    grid = F.affine_grid(
        theta, (1, C, H, W),
        align_corners=False)
    return F.grid_sample(
        sender_features, grid,
        align_corners=False)
```

The example's x coordinate points right and y points down. Translation is expressed in cells of this small feature grid. The factors $2/W$ and $2/H$ convert displacements to the normalized grid coordinates used by the sampler. The example call is

```
aligned_features = align_features(
    sender_features, angle_deg=90, dx=1, dy=1)
```

The original model's pose handling and scale conversion remain part of its implementation; this small example isolates the sampling operation.

A.3 Numerical Input and Kernel Dimensions

The separate convolution walkthrough uses a video tensor with three frames, three channels, height three, and width four. Figure 25 shows all values. Channel colors remain fixed across frames, making it possible to track a spatial patch through the numerical calculation.

The examples use stride one, no padding, zero bias, and one output channel. A 2×2 spatial kernel on a 3×4 plane produces a 2×3 output. A Conv3D kernel of depth two on three input frames also produces two output depth slices. Its complete output shape is therefore $2 \times 2 \times 3$ in depth–height–width order.

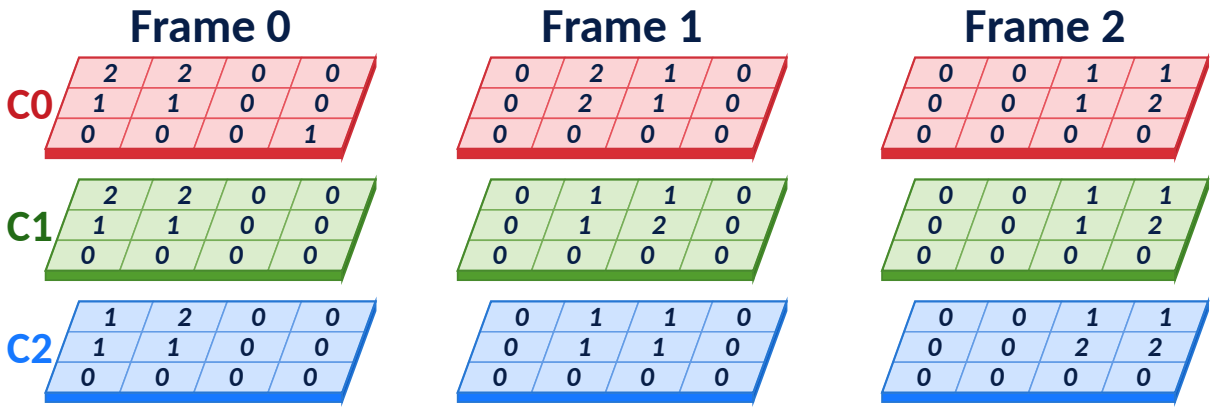


Figure 25: The project's numerical tensor example. Columns are frames; colors distinguish channels C0, C1, and C2. Each channel plane has three rows and four columns.

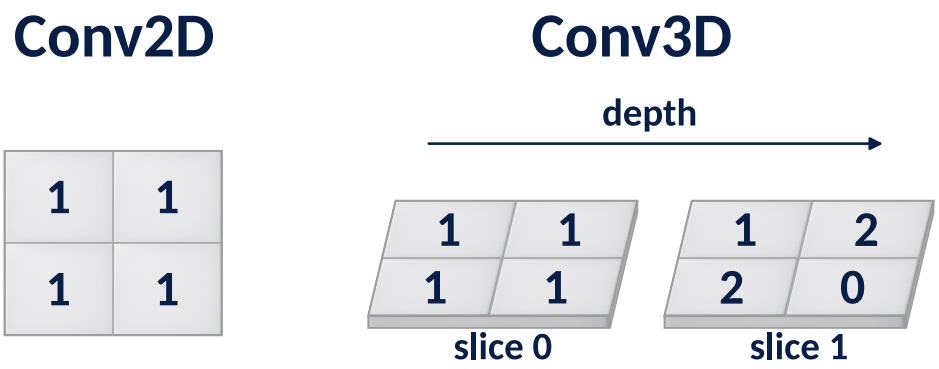


Figure 26: Project kernel illustrations. The Conv2D spatial kernel contains ones. The Conv3D kernel has two temporal slices, with different weights in the second slice. Each illustrated spatial/temporal kernel is applied across the input channels.

A.4 Conv2D: Accumulating Across Channels

For one output channel, the weight tensor contains a spatial kernel for every input channel. In this example all those Conv2D weights are one. At the upper-left output, the four-cell patches of Frame 0 sum to 6 in C0, 6 in C1, and 5 in C2. The contributions are added:

$$y_{0,0} = \underbrace{(2+2+1+1)}_{\text{C0}} + \underbrace{(2+2+1+1)}_{\text{C1}} + \underbrace{(1+2+1+1)}_{\text{C2}} = 17.$$

Sliding the same kernel across Frame 0 gives

$$\begin{bmatrix} 17 & 9 & 0 \\ 6 & 3 & 1 \end{bmatrix}.$$

The output is one channel because all three input-channel contributions feed the same filter. Multiple output channels would use additional learned filters. Convolving each frame separately does not mix the values of Frame 0 with those of Frame 1.

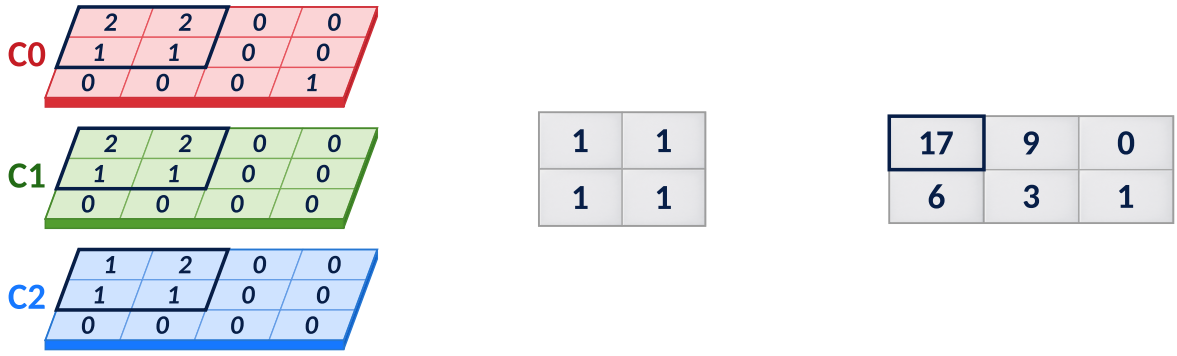


Figure 27: Project Conv2D example. Outlines select the upper-left patch in each channel; the highlighted output sums all channel contributions.

A.5 Conv3D: Accumulating Across Time and Channels

The temporal kernel in Figure 26 uses

$$K_0 = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \qquad K_1 = \begin{bmatrix} 1 & 2 \\ 2 & 0 \end{bmatrix}.$$

The same two-slice pattern is used for each input channel in this teaching example. At output $(d, r, c) = (0, 0, 0)$, K_0 multiplies the upper-left patch of Frame 0 and K_1 multiplies the corresponding patch of Frame 1. The recorded arithmetic is

$$(6 + 4)_{\mathrm{C0}} + (6 + 2)_{\mathrm{C1}} + (5 + 2)_{\mathrm{C2}} = 25.$$

This calculation combines both temporal slices and all three channels.

Advancing the depth coordinate moves the window to Frames 1 and 2. At $(1, 0, 0)$, the three channel contributions become $4 + 0$, $2 + 0$, and $2 + 0$, giving eight. The complete two output slices in the saved walkthrough are

$$Y_0 = \begin{bmatrix} 25 & 27 & 11 \\ 14 & 15 & 5 \end{bmatrix}, \qquad Y_1 = \begin{bmatrix} 8 & 21 & 24 \\ 4 & 16 & 20 \end{bmatrix}.$$

Depth and channels are separate axes: the kernel slides over time while accumulating over channels. This genuine multi-frame example explains the additional dimension of Conv3D. It should be distinguished from the architecture's pointwise helper on a singleton depth axis. The complete sequence of spatial and temporal windows remains in Appendix C of the presentation.

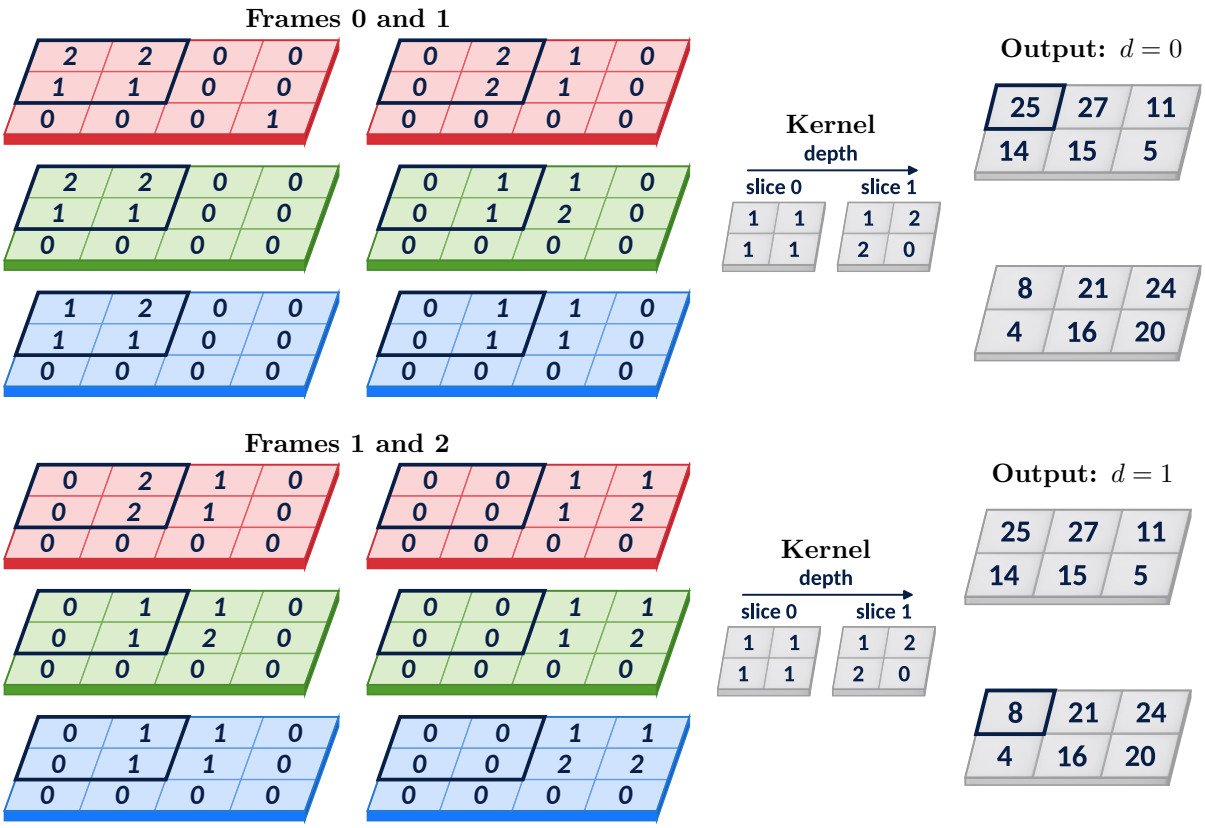


Figure 28: The project's input patches, temporal kernel, and output diagrams combined at their original aspect ratios. Above, Frames 0 and 1 produce 25 at $(0, 0, 0)$. Below, moving the temporal window to Frames 1 and 2 produces 8 at $(1, 0, 0)$.

References

[1] CoPerception Developers. V2X-Sim: Dataset and data preparation documentation. `https://coperception.readthedocs.io/en/latest/datasets/v2x_sim/`. Accessed November 8, 2026.

[2] Yiming Li, Dekun Ma, Ziyan An, Zixun Wang, Yiqi Zhong, Siheng Chen, and Chen Feng. V2X-Sim: Multi-agent collaborative perception dataset and benchmark for autonomous driving. *IEEE Robotics and Automation Letters*, 7(4):10914–10921, 2022. `https://arxiv.org/abs/2202.08449`. Accessed November 9, 2026.

[3] Yiming Li, Shunli Ren, Pengxiang Wu, Siheng Chen, Chen Feng, and Wenjun Zhang. Learning distilled collaboration graph for multi-agent perception. In *Advances in Neural Information Processing Systems*, 2021. `https://arxiv.org/abs/2111.00643`. Accessed November 9, 2026.

[4] Luis Alberto Vásquez Vargas. MO809: Cooperative perception with DiscoNet, 2s2026. Project repository. `https://github.com/LuisAlbertoVasquezVargas/MO809-Cooperative-Perception-2s2026`. Accessed November 9, 2026.